CREACIÓN DE CHATBOTS · SEGUNDO CORTE · EAM-2026II

AnimaBot Veterinaria

Chatbot con memoria, FastAPI y SQLite

Asistente virtual para consultar servicios y agendar, modificar y cancelar citas en una clínica veterinaria ficticia.

Equipo

Jeronimo Parra Hurtado · Backend y API

Sebastian Ospina Herrera · Prompt e interfaz

Jose Manuel Bravo Navarro · Pruebas y documentación

Todos los datos del proyecto son ficticios (práctica de clase).

Identificación y contexto

Negocio. Una clínica veterinaria (ficticia) que ofrece cinco servicios: consulta general, vacunación, desparasitación, peluquería y baño, y cirugía menor.

Usuarios. Clientes y tutores de mascotas que quieren informarse o pedir una cita sin esperar a que alguien conteste.

Proyecto. AnimaBot es un chatbot: un programa con el que se conversa por escrito, como en un chat, y que entiende lo que la persona pide.

Problemas que resuelve

- **Desinformación:** precios, duraciones y horarios no están a la mano.
- **Agendar a mano:** crear, mover y cancelar citas requiere atención humana permanente.
- **Pérdida de contexto:** si el cliente responde con una sola palabra («Arturo»), el sistema debe saber a qué pregunta responde.
- **Colisiones de horario:** dos citas no pueden ocupar el mismo momento.

> Idea central: el chatbot atiende 24/7, recuerda lo que la persona ya dijo y nunca guarda una cita que choque con otra.

Solución y alcance

Sí puede

- Informar servicios, precios, duraciones y horarios de atención.
- Mostrar las horas libres de un día para un servicio.
- Reservar, consultar, modificar y cancelar citas.
- Responder datos puntuales («¿a qué hora es mi cita?» → solo la hora).
- Recordar nombre del cliente, de la mascota, servicio, fecha y hora durante la conversación.
- Conservar esos datos aunque la persona escriba algo fuera de tema.

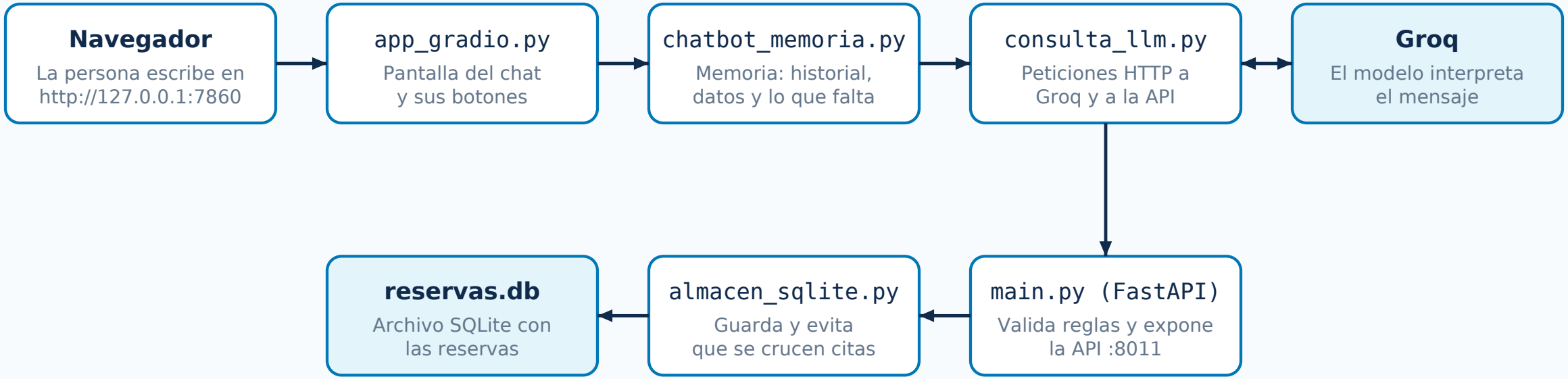
No puede

- Agendar en un horario ocupado o fuera de la jornada de atención.
- Asignar el mismo nombre al cliente y a la mascota por error.
- Ejecutar un cambio sin que la persona lo confirme antes.
- Borrar el historial: una cita cancelada queda guardada con estado «cancelada».
- Atender temas ajenos a la veterinaria: responde amablemente que solo ayuda con servicios y citas.

Las herramientas y para qué sirve cada una

Herramienta	Tarea dentro del proyecto
Python	Lenguaje en el que está escrito todo el programa; conecta las demás piezas.
Gradio	Biblioteca que crea la pantalla web del chat: cuadro de texto, botones Enviar, Confirmar y Descartar.
Groq (modelo de lenguaje)	Servicio de inteligencia artificial que interpreta el mensaje: qué quiere la persona y con qué datos (servicio, fecha, hora…). Modelo usado: `openai/gpt-oss-120b`.
FastAPI	API: un servicio que recibe órdenes del negocio (crear, consultar, modificar o cancelar una cita) y valida las reglas.
SQLite	Base de datos en un archivo (`reservas.db`) donde las citas quedan guardadas aunque se apague el sistema.
Flowise + JSON	Versión anterior del proyecto: flujo visual con archivos JSON como almacenamiento (ver «Evolución»).

Funcionamiento: cómo se conectan las piezas



Quién hace qué: el modelo de Groq solo interpreta lo que la persona escribe. Todo lo demás lo decide el programa: qué dato falta, si el horario está libre, el resumen que se muestra y los textos de respuesta. Por eso el chatbot responde siempre con el mismo estilo.

Separación de responsabilidades

Decisión	Razón	Alternativa descartada
Conversación y NLU separadas del negocio	El modelo solo interpreta intención y datos. La disponibilidad y el guardado los valida FastAPI/SQLite.	Dejar que el modelo modifique la base de datos directamente.
Persistencia en SQLite	Integridad de los datos y consultas estructuradas, sin corrupción por concurrencia.	Guardar las reservas solo en archivos JSON planos.
Consulta puntual por campo	La persona pregunta solo el precio, la hora o el estado sin leer la cita completa.	Devolver siempre la reserva completa.
Memoria de la conversación	Se conservan cliente, mascota y servicio aunque haya mensajes de una palabra o fuera de tema.	Reiniciar el diálogo ante cualquier respuesta inesperada.
Confirmar antes de guardar	Reservar, modificar y cancelar solo se ejecutan al pulsar «Confirmar operación».	Ejecutar el cambio en cuanto se entiende la petición.
Cancelar sin borrar	La cita cambia a estado «cancelada» y libera el horario, pero la fila se conserva.	Eliminar físicamente la fila.

Recorrido de uso: crear una reserva

Así conversa una persona con AnimaBot para reservar una vacunación. El chatbot pide solo lo que falta y la persona puede responder con una palabra.

#	La persona	El chatbot
1	Escribe: «¿Cuánto cuesta una vacunación?»	Vacunación: $45.000 COP, 20 minutos (solo ese servicio).
2	«Quiero una vacunación para mi perrita Princesa»	Resume `Llevo: Vacunación · mascota Princesa` y pregunta la fecha.
3	«el próximo lunes»	Lista los horarios libres y pregunta la hora.
4	«a las 10:30»	Pregunta: ¿a nombre de quién registro la cita?
5	«Laura»	Muestra un **resumen**: servicio, mascota, cliente, fecha, 10:30 a 10:50, $45.000. **Todavía no se guardó nada.**
6	Pulsa **Confirmar operación**	**Cita registrada** ✓ con un código como `AB-1A2B3C4D5E6F`. Aquí se guarda en SQLite.

Se comprueba dos veces: el chatbot revisa la disponibilidad antes del resumen y la base de datos la vuelve a revisar al confirmar.

Recorrido de uso: consultar, modificar y cancelar

Paso	La persona	El chatbot
7	«¿A qué hora es mi cita?»	Responde solo la hora: 10:30 y el código. «Mi cita» = la última de la conversación.
8–9	«Cámbiala para las 11:00» + Confirmar	Resumen del cambio 10:30 → 11:00; luego «Cita modificada ✓». El código no cambia.
10–11	«Cancela mi cita» + **Descartar solicitud**	No se ejecuta nada; la cita sigue activa.
12	«Cancela mi cita» + Confirmar	Cita cancelada. El horario quedó libre.
13	«¿Qué horas hay libres ese lunes?»	10:30 y 11:00 vuelven a aparecer como libres.
14	«Ahora una desparasitación el martes a las 9:00»	El resumen ya trae a Princesa y a Laura: la **memoria** evita preguntarlo otra vez.

Mientras hay una operación pendiente la pantalla no deja enviar otro mensaje: primero hay que confirmar o descartar. «Reiniciar conversación» borra la memoria, pero no las reservas ya guardadas.

Reglas del negocio

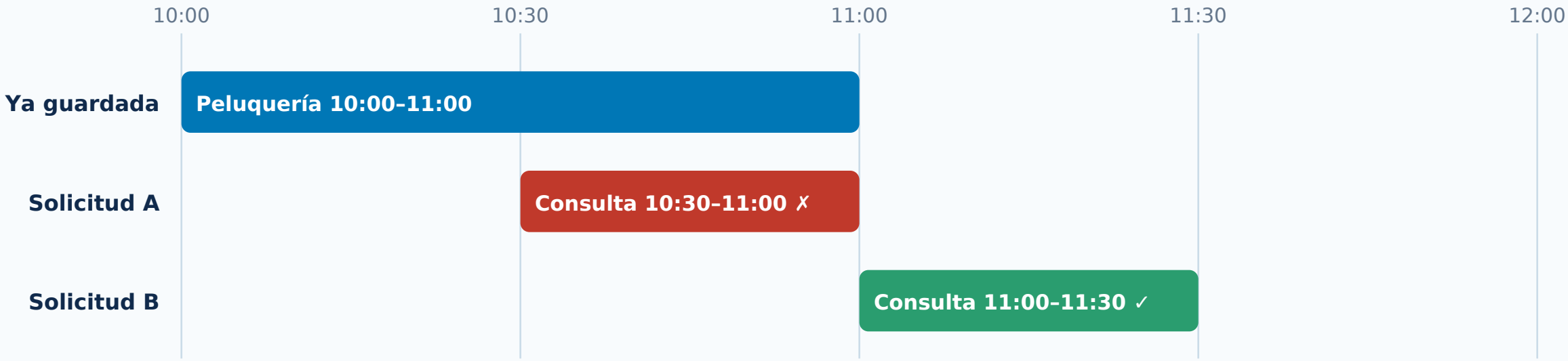
- **Horario:** lunes a viernes 08:00–18:00, sábado 08:00–13:00, domingo cerrado.
- **Pasos de 30 minutos:** las citas empiezan en :00 o :30.
- **La atención debe terminar dentro del horario**, así que la última hora de inicio depende de la duración.
- **Una sola agenda:** dos citas no pueden cruzarse, aunque sean de servicios distintos; una que empieza justo cuando otra termina sí se acepta.
- **Nada en el pasado:** ni fechas vencidas ni horas de hoy que ya empezaron.

Servicios y última hora de inicio

Servicio	Dur.	Precio COP	Últ. lun–vie	Últ. sáb
Consulta general	30 min	$60.000	17:30	12:30
Vacunación	20 min	$45.000	17:30	12:30
Desparasitación	20 min	$35.000	17:30	12:30
Peluquería y baño	60 min	$55.000	17:00	12:00
Cirugía menor	90 min	$280.000	16:30	11:30

Prevención de cruces: un ejemplo

Una peluquería dura 60 minutos. Si alguien la reserva el lunes a las 10:00, ocupa de 10:00 a 11:00. Veamos qué pasa con otras dos solicitudes:



Solicitud A (10:30) se solapa con la peluquería: el chatbot responde que esa hora no está disponible, muestra las horas libres y **no pide confirmación**. **Solicitud B** (11:00) empieza justo cuando la otra termina: se acepta.

API y persistencia en SQLite

La API (FastAPI) ofrece una operación por cada acción sobre las citas; `almacen_sqlite.py` las guarda en la tabla `reservas`.

Acción	Operación HTTP	Efecto en la base
Crear una cita	POST	Inserta una fila nueva con estado activa.
Consultar	GET	Lee una cita o la lista completa (incluye canceladas).
Modificar	PUT	Cambia fecha, hora, servicio, cliente o mascota; el código se mantiene.
Cancelar	DELETE	Cambia el estado a cancelada y libera la franja; la fila no se borra.

```
Fila de ejemplo en reservas.db
codigo: AB-1A2B3C4D5E6F
cliente: Laura
mascota: Princesa
servicio: vacunacion
fecha: 2030-10-14
hora: 10:30
estado: activa
(+ duracion_minutos)
```

Las reservas se conservan aunque se apague todo. Se pueden ver en `http://127.0.0.1:8011/reservas` o abriendo `reservas.db` con DB Browser for SQLite.

Pruebas y resultados

Script	Qué valida	Resultado esperado
`pruebas_almacen_sqlite.py`	Operaciones sobre la base: crear, consultar, modificar, cancelar y rechazar cruces.	19 pruebas: **OK**
`probar_demo.py`	Flujo completo de la API; cada caso cancela sus reservas al terminar (teardown) para no dejar datos basura.	4 líneas **OK**
`pruebas_chatbot.py`	Extracción de datos, memoria con respuestas de una palabra y mensajes fuera de tema.	9 líneas **OK**; última: «chatbot completo»

Las pruebas **no necesitan clave de Groq ni internet** y usan una base de datos temporal, por lo que no tocan las reservas reales.

Comportamientos que se verifican: memoria con mensajes de una palabra y fuera de tema · crear, modificar y cancelar · rechazo de horarios ocupados · liberación del horario al cancelar · modificar una cita sin que se bloquee a sí misma.

Problemas encontrados y cómo se resolvieron

Problema	Causa	Solución	Verificación
Cliente y mascota con el mismo nombre	Ante una entrada ambigua («Carlos») el extractor llenaba ambos datos.	Reglas más estrictas en el prompt y validación en el código para asignar cada nombre una sola vez.	`pruebas_chatbot.py` con entradas ambiguas.
Pérdida de contexto fuera de tema	El mensaje se clasificaba como «fuera de tema» y se ignoraba lo acumulado.	Se conserva el diccionario de la sesión en intenciones que no son operaciones.	Prueba que intercala preguntas ajenas durante el agendamiento.
Reservas basura tras las pruebas	Las pruebas creaban citas y no las limpiaban.	Se cancela cada reserva en el bloque final (teardown) de cada caso.	Revisión de SQLite tras correr todo.
Modificar una cita se bloqueaba a sí misma	Al cambiar la hora, la cita actual contaba como horario ocupado.	El PUT excluye la reserva actual al calcular disponibilidad.	Cambios de hora sobre la misma reserva.

Evolución: de Flowise y JSON a Python, Gradio y SQLite

Aspecto	Versión con Flowise y JSON	Versión del segundo corte
Interpretación del mensaje	Grafo y prompts en Flowise (`Demo_03_Flowise_FastAPI.json`) con un modelo de Groq.	Código en Python (`consulta_llm.py`): prompt, historial y llamadas al modelo.
Interfaz	Flowise.	Pantalla propia en Gradio (`app_gradio.py`) con Confirmar, Descartar y Reiniciar.
Almacenamiento	Archivos JSON (`reservas.json`).	Base SQLite con validación y control de cruces (`almacen_sqlite.py`).
Memoria y confirmación	—	Estado por sesión en `chatbot_memoria.py` y confirmación antes de cambiar datos.
Modelo	`llama-3.3-70b-versatile`, retirado por Groq el 16 de agosto de 2026.	`openai/gpt-oss-120b`, configurable con `GROQ_MODEL`.

El equipo asumió en el código lo que antes hacía la herramienta: la memoria, las validaciones, el control de horarios y los textos de respuesta.

Cierre: limitaciones y repositorio

Limitaciones actuales

- Las fechas relativas («el viernes») las interpreta el modelo; si entiende otro día, hay que escribir la fecha completa.
- Cada cita admite un solo servicio y la búsqueda es por código, no por nombre de mascota.
- La memoria vive en la sesión: al reiniciar o recargar se pierde (las reservas sí se conservan).
- Sistema de práctica: datos ficticios y ejecución local.

Documentos del proyecto

- `README.md`: instalación, configuración y ejecución en dos terminales.
- `docs/Bitacora_Segundo_Corte.md`: documentación principal del corte.
- `docs/Presentacion_Segundo_Corte.pdf`: este documento.
- `docs/evidencias/`: capturas y pruebas.

Repositorio en GitHub:

`https://github.com/parraJeronimo/chat_bot`